PRÁCTICA 6

Atascos por Salto. Reducción con BTB y Delay Slot. Atascos Estructurales y WAR/WAW.

Parte 1: Reducción de Atascos por Salto con BTB

1) Repaso de atascos por salto (BTS) ⭐

```
.data
A:              .word 1
B:              .word 3
C:          .word 0
.code
        ld      $t1, A($0)
        ld      $t2, B($0)
loop:   dsll    $t1, $t1, 1
        daddi   $t2, $t2, -1
        bnez    $t2, loop
        sd $t1, C($0)
        halt
```

Ejecutar el programa anterior con Forwarding, de modo que no haya atascos RAW. El programa sigue teniendo algunos atascos, llamados Branch Taken Stall (BTS), también conocidos como atascos por salto.

1. Cambiar el valor de B a 1000 y ejecutar. ¿Cómo cambió la cantidad de BTS? ¿y el CPI?
2. Completar la siguiente frase: Al ejecutar un loop simple con N iteraciones, se producen N-1 atascos BTS.
3. ¿Por qué se producen los BTS? ¿Qué sucede con la instrucción siguiente al salto?
4. ¿Pueden evitarse los BTS reordenando instrucciones?
5. Al ocurrir un BTS, una instrucción se empieza a ejecutar y luego se descarta, pero ¿debe contarse como una instrucción para el cálculo del CPI?

2) Reducción de BTS con Branch Target Buffer ⭐

Habilitar la opción Branch Target Buffer (BTB) y volver a ejecutar el programa anterior con B=3.

1. ¿Cómo cambió la cantidad de BTS? ¿y el CPI?
2. Notar que ahora aparece un nuevo tipo de atasco, llamado Branch Misprediction Stall (BMS). ¿Por qué sucede? ¿Cuántos ocurren?
3. Cambiar el valor de B a 1000, habilitar BTB y ejecutar. ¿Cómo cambió la cantidad de BTS y BMS? ¿y el CPI?
4. Completar la siguiente frase: Al ejecutar un loop simple con N iteraciones, si se habilita BTB, se producen 2 atascos de tipo BTS y 2 atascos de tipo BMS.
5. En base a los resultados anteriores ¿es mejor utilizar BTB cuando se realizan pocas o cuando se realizan muchas iteraciones?

3) Utilidad del BTB en distintos casos ⭐

El BTB puede aumentar el desempeño significativamente en la mayoría de los lazos. No obstante, en algunos puede tener un comportamiento patológico y de hecho reducir la eficiencia del programa. El siguiente programa calcula el máximo de un vector.

```
.data
A:            .word 2,1,3,1,4,1
MAX: .word -1
.code
       ld $t1, MAX($0)
       daddi $t2,$0,0
       daddi $t3,$0,6
```

```
loop: ld $t4, A($t2)
      slt $t5,$t1,$t4
      beqz $t5, chico
      daddi $t1,$t4,0
chico:daddi $t2,$t2,8
      daddi $t3, $t3, -1
      bnez  $t3, loop
      sd $t1, MAX($0)
      halt
```

a) Antes de ejecutar en el simulador, encontrar las instrucciones de salto, y pensar cómo se comportará el BTB en cada caso. Recordar que la predicción del BTB guarda un bit de historia distinto por cada instrucción de salto.
b) Ejecutar el programa en el simulador con y sin BTB. ¿Qué programa es más eficiente?

Parte 2: Reducción de Atascos por Salto con Delay Slot

1) Reducción de BTS con Delay Slot (DS) ⭐⭐

```
.data
A:              .word 1
B:              .word 3
C:          .word 0
.code
        ld      $t1, A($0)
        ld      $t2, B($0)
loop: dsll $t1, $t1, 1
        daddi   $t2, $t2, -1
        bnez    $t2, loop
        sd $t1, C($0)
        halt
```

El delay slot (DS) cambia el funcionamiento de los saltos. Cuando está activado, el salto se realiza con un retardo de un ciclo. Esto significa que la instrucción **siguiente** al salto también se ejecuta. De esta forma, los BTS desaparecen completamente, aunque esto no significa necesariamente que la ejecución sea más eficiente.

a) Anotar en la tabla de abajo las estadísticas del programa al ejecutarse sin forwarding, y con **BTB** y **delay slot** desactivados
b) Ejecutar el programa, pero ahora con BTB activado y forwarding desactivado.
c) Ejecutar el programa, pero ahora con **delay slot** activado, BTB desactivado y forwarding activado. Anotar los CPI y la cantidad de instrucciones en la tabla de abajo. ¿Qué sucede con la instrucción **sd $t1, C($0)**? ¿Cuántas veces se ejecuta? ¿Son necesarias todas las ejecuciones?
d) Modificar el programa agregando un NOP antes de la instrucción **sd $t1, C($0)**. Ejecutar el programa en el simulador y anotar los CPI. Notar que si bien el CPI puede bajar, también subió la cantidad de instrucciones a ejecutar, y los ciclos totales usados.
e) Modificar el programa, pero ahora reordenando las instrucciones de modo de ejecutar una instrucción **útil** debajo del salto, sin aumentar el número total de instrucciones a ejecutar.

Ejercicio 3	Sin mejoras	BTB	DS	DS + NOP	DS + Reordenamiento
BTS	4	2	0	0	0
BMS	0	2	0	0	0
CPI	2	28/13=2,154	26/15 =1,733	27/16=1,68	2
#Instrucciones	13	13	15	16	13

2) Corrección de programas con Delay Slot ⭐⭐⭐

En el programa anterior, habilitar DS hace que una instrucción se ejecute varias veces. Si bien el programa es más ineficiente, el resultado final es el mismo. No obstante, en otros casos habilitar DS puede hacer que el programa no funcione correctamente. Por ejemplo, analizar el siguiente programa:

```
.data
A:              .word 2,1,3,1,4,1
MAX: .word -1
.code
       ld $t1, MAX($0)
       daddi $t2,$0,0
       daddi $t3,$0,6
```

```
loop: ld $t4, A($t2)
      slt $t5,$t1,$t4
      beqz $t5, chico
      daddi $t1,$t4,0
chico:daddi $t2,$t2,8
      daddi $t3, $t3, -1
      bnez  $t3, loop
      sd $t1, MAX($0)
      halt
```

a) Correr el programa con DS habilitado y anotar el CPI. Notar el resultado almacenado en la variable MAX ¿Por qué es incorrecto?
b) Reordenar las instrucciones del programa para que funcione correctamente, y para que además la instrucción **sd $t1, MAX($0)** solo se ejecute una vez, sin agregar instrucciones a ejecutar. Anotar el CPI, y comparar con el caso anterior ¿qué mejora se obtuvo?

Parte 3: Atascos por WAR, WAW y estructurales.

La etapa EX usa la ALU de sumas y restas de punto fijo. En un programa que solo utiliza instrucciones que pasan por la etapa EX, las instrucciones no se pueden *sobrepasar*, ya que todas tardan el mismo tiempo y pasan por las mismas etapas. En los procesadores modernos, no obstante, existen distintas ALUs para las operaciones de suma/resta (EX), multiplicación (MUL) y división (DIV). Además, las instrucciones de multiplicación y división tardan más ciclos que las de suma. Por este motivo, las instrucciones de suma pueden **comenzar después** que las de multiplicación o división, y **terminar antes,** generando efectivamente una **ejecución fuera de orden** (out of order execution).
Este tipo de ejecución abre la posibilidad a tres nuevos tipos de atascos: WAR (Write After Read, o **atascar la escritura para que termine la lectura**), WAW (Write After Write, o **atascar la escritura para que termine la otra escritura**) y STR (estructurales, o **atascar una instrucción porque la siguiente etapa o estructura del procesador está siendo utilizada**). NOTA: usar ***forwarding activado*** para los siguientes ejercicios.

1) Atascos WAR y WAW ⭐

Los atascos WAR y WAW son la contracara de los RAW. Si bien es mucho más difícil que se produzcan en un programa común, son posibilidades que debe tener en cuenta el procesador. Estos atascos suceden cuando una **instrucción más rápida** se adelanta a una **instrucción más lenta**. Los siguientes programas presentan ejemplos minimalistas de estos tipos de atascos.

1. Estudiar el código sin simularlo y responder ¿cuál es el programa que tiene WAR y cual WAW? Simular los programas para comprobar.
2. En el caso del WAR, ¿cuál es la instrucción lenta y cuál la rápida? ¿Qué registro se quiere leer y escribir?
3. Idem para el caso WAW.

```
.code
ddiv $t1, $t2, $t3
dadd $t1, $t2, $t3
halt
```

```
.code
dmul $t1, $0, $0
dmul $t3, $t1, $t2
dadd $t2, $0, $0
halt
```

2) Atascos estructurales (STR) ⭐

La ejecución fuera de orden también permite otro tipo de atasco. Los atascos estructurales suceden cuando dos instrucciones quieren acceder al mismo tiempo a la misma etapa del cauce. En el simulador, esto sucede en la etapa MEM, ya que sin importar la ALU que usen las instrucciones, luego del cálculo siempre deben pasar por la etapa MEM. Como las instrucciones rápidas pueden sobrepasar a las lentas, puede suceder que dos o más instrucciones terminen su etapa de ejecución al mismo tiempo, y por ende también quieran pasar a MEM al mismo tiempo.

```
.code
dmul $t1, $0, $0
nop
nop
nop
```

```
nop
nop
dadd $t2,$0,$0
halt
```

a) Ejecutar el código anterior con *forwarding activado* y verificar que ocurre un atasco STR. ¿Entre qué instrucciones sucede el atasco? ¿cuál es la instrucción que se atasca? ¿Por qué esa y no la otra?
b) Probar agregando un NOP ¿sigue el atasco? ¿y si se quita un NOP? ¿por qué?

3) Análisis de atascos ⭐⭐

Los siguientes programas presentan ejemplos naturales de atascos estructurales y WAR. Analizar e identificar dónde pueden ocurrir estos atascos. Ejecutar en el simulador con *forwarding activado* y comprobar el resultado.

```
; Resto: Calcula en $t4 el resto
; de $t1 div $t2
.code
daddi $t1,$0,30 ; a = 30
daddi $t2,$0,4  ; b = 4
ddiv  $t3,$t1,$t2 ; c = a div b = 7
dmul $t3, $t3, $t2 ; c*b = 7*4 = 28
dsub $t4, $t1,$t3 ; resto = a-c*b = 2
halt
```

```
; factorial: Calcula en $t2
; el factorial de $t1
   .code
        daddi $t1,$0,5 ; n=5
        daddi $t2,$0,1 ; f=1
   loop: dmul $t2,$t2,$t1 ; f=f*n
        daddi $t1,$t1,-1 ; n=n-1
        bnez $t1, loop
        halt
```

4) Etapas y atascos ⭐⭐

Completar las etapas en la ejecución de los siguientes programas, asumiendo forwarding activado.

a) Suma y producto

Ciclo	1	2	3	4	5	6	7	8	9	11	11	12	13	14	15	16	17	18
ld $t1, A($0)	IF	ID	EX	ME	WB													
ld $t2, B($0)		IF	ID	EX	ME	WB												
dmul $t3,$t1,$t2			IF	ID	m0	raw	m1	m2	m3	m4	m5	m6	me	wb				
sd $t3, MULT($0)				IF	RAW	ID	ex	raw	raw	raw	raw	raw	str	me	wb			
dadd $t3,$t1,$t2					IF		id	id	id	id	id	id	id	ex	me	wb		
sd $t3, SUMA($0)							IF							id	EX	ME	WB	
halt														IF	ID	EX	ME	WB

b) Promedio

Ciclo	1	2	3	4	5	6	7	8	9	11	11	12	13	14	15	16	17
ld $t1, suma($0)	IF	ID	EX	ME	WB												
ld $t2, cant($0)		IF	ID	EX	ME	WB											
ddiv $t3,$t1,$t2			IF	ID	div	raw	--	--	--	--	--	--		--	--	--	
sd $t3, prom($0)				IF	id	raw	ex	raw	raw	raw	raw	raw	raw	raw	raw	raw	
halt					F		ID										

Ciclo	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
ld $t1, suma($0)																
ld $t2, cant($0)																
ddiv $t3,$t1,$t2												div	ME	WB		
sd $t3, prom($0)	raw	raw	raw	raw	raw	raw	raw	raw	raw	raw	raw	raw	str	ME	WB	
halt														EX	ME	WB

Parte 4: Ejercicios de repaso o tipo parcial

1) Análisis de atascos con lazos ⭐⭐⭐

El siguiente programa calcula la suma del arreglo A y almacena el resultado en la variable SUM. Calcular la cantidad de instrucciones, atascos y ciclos totales que toma el programa, y los CPIs, sin utilizar el simulador, asumiendo BTB/DS desactivados, y:

a) Asumiendo **forwarding desactivado**
b) Asumiendo **forwarding activado**.
c) En el caso a), ¿qué sucedería si la instrucción daddi $t2,$0,0 se intercambia por daddi $t3,$0,3?

Verificar los resultados con el simulador en cada caso.

```
.data
A:            .word 2,1,3
SUM: .word 0

.code
       daddi    $t2, $0, 0
       ld       $t1, SUM($0)
       daddi    $t3, $0, 3
```

```
loop: ld        $t4, A($t2)
      dadd      $t1, $t1, $t4
      daddi   $t3, $t3, -1
      bnez    $t3, loop
      sd        $t1, SUM($0)
      halt
```

Pista: Es posible simular manualmente la ejecución de todo el programa ciclo por ciclo, pero esto resulta engorroso, sobre todo para los lazos con varias iteraciones. En lugar de eso, analizar el programa en 3 partes separadas: el código antes del loop, el código del loop (que se repite 3 veces) y el código después del loop. Por cada parte, determinar la cantidad de instrucciones ejecutadas. Para calcular los ciclos, primero determinar las dependencias de datos, en base a eso los atascos, y en base a eso y utilizar la fórmula vista anteriormente para calcular la cantidad de ciclos totales.

2) Cálculo de CPI con lazo tipo while y con forwarding ⭐⭐⭐

El siguiente programa busca determinar si el número **num** se encuentra dentro del vector **tabla**.

A. Calcular manualmente el número de ciclos, CPI, RAWs y BTS/BMS, asumiendo forwarding activado y BTB desactivado.
B. Idem, asumiendo BTB activado.
C. Modificar el programa para que con DS activado funcione correctamente y no ejecute instrucciones de más.

```
.data
  tabla: .word 20, 1, 14, 7, 12, 11
  num:          .word 7
  long:         .word 6
 res: .word 0
.code
               ld      $t1, long($0)
               ld      $t2, num($0)
               dadd    $t3, $0, $0
               dadd    $t10, $0, $0
```

```
loop:          ld      $t4, tabla($t3)
               beq     $t4, $t2, listo
               daddi   $t1, $t1, -1
               daddi   $t3, $t3, 8
               bnez    $t1, loop
               j       fin
listo:         daddi   $t10, $0, 1
fin:           sd $t10, res($0)
          halt
```

3) Cálculo de CPI con lazo tipo for y sin forwarding ⭐⭐⭐

El siguiente programa multiplica por 2 los valores del vector datos mediante un desplazamiento a la izquierda (**dsll**).

A. Calcular manualmente el número de ciclos, CPI, RAWs y BTS/BMS, asumiendo **forwarding desactivado**, **BTB/DS desactivados.**

B. Modificar el programa para funcionar correctamente con DS. Calcular manualmente el número de ciclos, CPI, RAWs y BTS/BMS, asumiendo **forwarding desactivado** y **DS activado**.

```
.data
cant: .word 8
datos: .word 1, 2, 3, 4, 5, 6, 7, 8
res: .word 0
.code
dadd $t1, $0, $0
ld $t2, cant($0)
```

```
loop: ld $t3, datos($t1)
      daddi   $t2, $t2, -1
      dsll    $t3, $t3, 1
      sd      $t3, res($t1)
      daddi   $t1, $t1, 8
      bnez    $t2, loop
      halt
```